

---

## What Is Manual Scheduling?

In Kubernetes, **scheduling** refers to the process of **deciding which node** a pod should run on.

Normally, the **Kubernetes scheduler** does this automatically, considering things like:

- available CPU and memory 🧠
- taints and tolerations ☢️
- affinity/anti-affinity rules 🤝
- node selectors 🎯

💡 But with manual scheduling, you bypass the scheduler and tell Kubernetes exactly which node the pod should run on.

---

## How Manual Scheduling Works

You specify a pod and directly assign it to a node using the `nodeName` field.

📌 This is a **hard assignment**, meaning:

- the pod is scheduled **only** to the node you specify
- if that node is down or unschedulable, the pod will **stay pending** ❌

# Learning Objectives

By the end of this lab, students will be able to:

1. Generate SSH keys and register them with kOps
  2. SSH into control-plane nodes
  3. Stop kube-scheduler safely
  4. Deploy pods **with** and **without** manual scheduling (`nodeName`)
  5. Observe the behavior differences and implications
  6. Restore the scheduler and clean up pods
- 

## Step 1: Generate SSH Keys for Cluster Access

Check if SSH keys exist:

```
ls ~/.ssh
```

If `id_rsa` and `id_rsa.pub` do **not** exist, generate a new key pair:

```
ssh-keygen -t rsa -b 4096 -C "ubuntu@kops-agent"
```

- Press **Enter** to accept default location (`~/.ssh/id_rsa`)
- Optional: set a passphrase

Verify the keys:

```
ls ~/.ssh/id_rsa ~/.ssh/id_rsa.pub
```

 Expected: Both private (`id_rsa`) and public (`id_rsa.pub`) keys exist.

---

## Step 2: Add Public Key to kOps Cluster

Register the public key:

```
kops create secret sshpublickey admin -i ~/.ssh/id_rsa.pub --name test.k8s.local
```

Apply the update:

```
kops update cluster --name test.k8s.local --yes
```

✓ This ensures your SSH key is deployed to all control-plane and worker nodes.

---

## Step 3: SSH into the Control-Plane Node

Get control-plane external IP:

```
kubectl get nodes -o wide
```

Example output:

| NAME                | ROLE          | INTERNAL-IP   | EXTERNAL-IP    |
|---------------------|---------------|---------------|----------------|
| i-01c56b5e21cd49577 | control-plane | 172.20.53.242 | 44.214.134.144 |

SSH into the control-plane:

```
ssh -i ~/.ssh/id_rsa ubuntu@44.214.134.144
```

List the static pod manifests:

```
ls /etc/kubernetes/manifests/
```

Expected output:

```
etcd-events-control-plane-us-east-1a.manifest
etcd-main-control-plane-us-east-1a.manifest
kube-apiserver.manifest
kube-controller-manager.manifest
kube-scheduler.manifest
```

---

## Step 4: Stop kube-scheduler (Demo Purpose)

Move the scheduler manifest temporarily:

```
sudo mv /etc/kubernetes/manifests/kube-scheduler.manifest /root/
```

Verify that the scheduler pod is stopped:

```
kubectl get pods -n kube-system | grep scheduler
```

✓ Expected: No output — scheduler pod terminated.

**Teaching Note:** Pods without `nodeName` will now **stay Pending** because the scheduler is not available.

---

## Step 5: Deploy Pod WITHOUT Manual Scheduling

Create `pod-no-scheduler.yaml`:

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-no-scheduler
```

```
spec:
  containers:
  - name: nginx
    image: nginx
```

Apply the pod:

```
kubectl apply -f pod-no-scheduler.yaml
```

Check pod status:

```
kubectl get pods
kubectl describe pod pod-no-scheduler
```

✓ Expected: Pod is **Pending**, with **FailedScheduling** events.

**Observation:** Scheduler is required for normal pod placement.

---

## Step 6: Deploy Pod WITH Manual Scheduling

Pick a node (from `kubectl get nodes -o wide`), e.g., `i-0818509055363be18`.

Create `pod-manual.yaml`:

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-manual
spec:
  containers:
  - name: nginx
    image: nginx
```

nodeName: i-0818509055363be18

Apply the pod:

```
kubecttl apply -f pod-manual.yaml
```

Check pod status:

```
kubecttl get pods -o wide
```

✅ Expected: Pod `pod-manual1` runs **successfully** on `i-0818509055363be18`

**Observation:** `nodeName` allows manual scheduling, bypassing the scheduler.

---

## Step 7: Restore kube-scheduler

Move the scheduler manifest back:

```
sudo mv /root/kube-scheduler.manifest /etc/kubernetes/manifests/
```

Verify scheduler pod:

```
kubecttl get pods -n kube-system | grep scheduler
```

✅ Scheduler pod should be **Running** again.

---

## Step 8: Clean Up Demo Pods

```
kubecttl delete pod pod-no-scheduler
```

kubectl delete pod pod-manual

✓ Cluster is back to normal.

---

## Step 9: Observations Table

| Pod Name         | Scheduler Running? | nodeName Specified? | Expected Behavior                         |
|------------------|--------------------|---------------------|-------------------------------------------|
| pod-no-scheduler | ✗                  | No                  | Pending (FailedScheduling)                |
| pod-manual       | ✗                  | Yes                 | Running on specified node                 |
| pod-manual       | ✓                  | Yes                 | Running on specified node                 |
| pod-no-scheduler | ✓                  | No                  | Scheduled automatically by kube-scheduler |

---

## Step 10: Key Teaching Points

- **nodeName** → manual scheduling, **bypasses the scheduler**
  - Scheduler is required only for pods **without nodeName**
  - Manual-scheduled pods have **no fault tolerance**
  - Stopping scheduler is safe for demo if using static pods
  - Scheduler automatically restarts when manifest is restored
-